Mini-project: ship passengers EDA

🎯 Today's goal

Today there's no new big topic. You'll use everything from Days 9–14 on one dataset, start to finish:

- write down **questions** before touching the data
- **inspect** and **clean** a passenger list (missing ages, duplicated rows)
- work out **survival rates** overall, by sex, by class and by age group
- draw three **charts** that answer the questions at a glance
- write your **findings** in plain English

🤔 Why this matters

One of the most famous beginner datasets in data science is the passenger list of the *Titanic*, the ship that sank in 1912. Who survived, and why? Today we work on a dataset **inspired by it**: the passengers of the *Northern Star*, a ship that never existed. **Every passenger and number below is made up** by a few lines of Python, so the results are about our invented ship, not about real people.

A mini-project is like a recipe that uses everything in your kitchen. You already know each step (pandas, cleaning, grouping, charts, EDA); today you see how they fit together into one story. On **Day 22** you'll come back to this same ship and teach a computer to *predict* who survives.

❓ Step 1: Questions first

Good analysis starts with questions, not with code. Here are ours for the *Northern Star*:

1. How many passengers survived overall?
2. Did women and men survive at different rates?
3. Did first-class passengers (the most expensive tickets) do better?
4. Were children more likely to survive?
5. Which group did best, and which did worst?

🚢 Step 2: Meet the data

This box **creates** the made-up passenger list with NumPy's random numbers (Day 9) and puts it in a DataFrame called `ships`. You don't need to understand every line; the important part is the table it builds. The seed `42` makes the "random" data the same every time, for everyone.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass == 2)
+ 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)

ships = pd.DataFrame({
    "passenger_id": np.arange(1, n + 1),
    "pclass": pclass,
    "sex": sex,
    "age": age,
    "fare": fare,
    "embarked": embarked,
    "survived": survived,
})
# Deliberate glitches to clean: 30 missing ages and 3 duplicated rows
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)

ships.head()
```

	passenger_id	pclass	sex	age	fare	embarked	survived
0	1	3	female	52.0	9.06	S	1
1	2	2	male	48.0	23.24	S	0
2	3	3	male	33.0	10.42	C	1
3	4	3	female	48.0	16.79	S	0
4	5	1	male	30.0	82.88	Q	0

What the columns mean:

Column	Meaning
`passenger_id`	a number for each passenger
`pclass`	ticket class: 1 = first (most expensive), 2 = second, 3 = third (cheapest)
`sex`	`"female"` or `"male"`
`age`	age in years (some are missing)
`fare`	ticket price in made-up pounds
`embarked`	the port where they got on: S, C or Q
`survived`	`1` = survived, `0` = did not survive

Honest note: we planted the glitches on purpose (the last lines of the box): 30 ages were deleted and 3 rows were copied twice. Real data comes with glitches like these, so finding them is part of the job.

🔍 Step 3: Inspect

Before any calculation, look at the size, the types, the missing values and the duplicates, just like the EDA routine from Day 14:

```
print("Rows and columns:", ships.shape)
print()
print("Missing values per column:")
print(ships.isna().sum())
print()
print("Duplicated rows:", ships.duplicated().sum())
```

```
Rows and columns: (403, 7)

Missing values per column:
passenger_id     0
pclass           0
sex              0
age             30
fare             0
embarked         0
survived         0
dtype: int64

Duplicated rows: 3
```

`describe()` gives a quick summary of every number column:

```
ships.describe().round(1)
```

	passenger_id	pclass	age	fare	survived
count	403.0	403.0	373.0	403.0	403.0
mean	199.2	2.2	30.4	33.4	0.4
std	116.2	0.8	12.4	34.8	0.5
min	1.0	1.0	1.0	4.0	0.0
25%	98.5	1.0	23.0	10.5	0.0
50%	199.0	2.0	30.0	16.3	0.0
75%	299.5	3.0	38.0	38.4	1.0
max	400.0	3.0	71.0	142.8	1.0

Look at the `count` row: `age` has fewer values than the other columns (those are the missing ones). The `mean` of `survived` is already a hint for question 1. Ages go from 1 to 71 and fares look sensible, so there are no impossible values to fix.

🧹 Step 4: Clean

Two fixes, both from Day 12. First remove the duplicated rows. Then fill the missing ages with the **median** age (the middle value), which isn't pulled around by a few very old or very young passengers. We keep `ships` as it was and store the result in a new DataFrame called `clean`:

```
clean = ships.drop_duplicates()

median_age = clean["age"].median()
clean["age"] = clean["age"].fillna(median_age)

print("Median age used for the gaps:", median_age)
print("Rows now:", len(clean))
print("Missing ages now:", clean["age"].isna().sum())
```

```
Median age used for the gaps: 30.0
Rows now: 400
Missing ages now: 0
```

Is filling ages fair? It's a guess, and a guess changes the data a little. The other choice is to drop those 30 passengers, but then we'd lose their sex, class and survival too. Always write down which choice you made, so others know.

📊 Step 5: Survival rates

Here's a neat trick. `survived` holds only 0s and 1s, so its **mean** is the share of 1s. For example, the mean of `[1, 0, 1, 1]` is 3 ÷ 4 = 0.75, which means 75% survived. That share is called the **survival rate**.

```
overall_rate = clean["survived"].mean()
print(f"Overall survival rate: {overall_rate:.0%}")
print("Survivors:", clean["survived"].sum(), "of", len(clean))
```

```
Overall survival rate: 41%
Survivors: 164 of 400
```

`:.0%` in an f-string turns a share like `0.41` into a percentage with 0 decimals: `41%`. Now question 2: group by `sex` and take the mean (Day 11):

```
rate_by_sex = clean.groupby("sex")["survived"].mean()
rate_by_sex.round(2)
```

```
sex
female    0.65
male      0.24
Name: survived, dtype: float64
```

A big difference: about two out of three women survived, but only about one in four men. Question 3 is about class. We also look at the average fare, to check that first class really was the expensive one:

```
by_class = clean.groupby("pclass").agg(
    passengers=("survived", "count"),
    survival_rate=("survived", "mean"),
    average_fare=("fare", "mean"),
)
by_class.round(2)
```

	passengers	survival_rate	average_fare
pclass			
1	103	0.54	85.73
2	99	0.36	24.12
3	198	0.36	10.80

Column names must match exactly. There is no column called `class` (it's `pclass`), so this gives a `KeyError`:

💡 This example has a mistake on purpose. Run it and read the error.

```
clean.groupby("class")["survived"].mean()
```

⚠️ KeyError

That key or column name doesn't exist. Check the spelling.

```
Traceback (most recent call last):
  line 1, in <module>
    clean.groupby("class")["survived"].mean()
    ~~~~~~~~~~~~~~^^^^^^^^^
KeyError: 'class'
```

When you meet a `KeyError` in pandas, print `clean.columns` and compare the spelling.

👶 Age groups with `pd.cut`

Question 4 is about children. Age is a number with many different values, so comparing every single age would be messy. Instead we put ages into **bins** (ranges), like sorting coins into jars. `pd.cut` does this. You give it the edges of the bins and a label for each one:

```
clean["age_group"] = pd.cut(
    clean["age"],
    bins=[0, 12, 18, 40, 60, 80],
    labels=["child", "teen", "adult", "middle-aged", "senior"],
    right=False,    # "child" = 0 up to (not including) 12
)
clean[["age", "age_group"]].head(6)
```

	age	age_group
0	52.0	middle-aged
1	48.0	middle-aged
2	33.0	adult
3	48.0	middle-aged
4	30.0	adult
5	33.0	adult

With `right=False`, the bin `child` means "0 or more, but under 12", `teen` means 12 to 17, and so on. Now group by the new column:

```
by_age = (clean.groupby("age_group", observed=True)["survived"]
          .agg(["count", "mean"]))
by_age.round(2)
```

	count	mean
age_group		
child	24	0.62
teen	36	0.44
adult	259	0.39
middle-aged	77	0.42
senior	4	0.25

`observed=True` tells pandas to show only the bins that really have passengers. Children did best. But look at the `count` column too: only 4 seniors, so their rate could easily change with a few more passengers. **Small groups give shaky rates.**

🧮 Sex and class together

Question 5 needs two columns at once. A `pivot_table` (Day 11) puts sex in the rows, class in the columns, and the survival rate in each cell:

```
table = clean.pivot_table(index="sex", columns="pclass",
                          values="survived", aggfunc="mean")
table.round(2)
```

pclass	1	2	3
sex			
female	0.77	0.62	0.6
male	0.36	0.21	0.2

Reading the table: first-class women did best (about 77%), and second- and third-class men did worst (about 20%). Sex made a bigger difference than class.

📈 Step 6: Charts

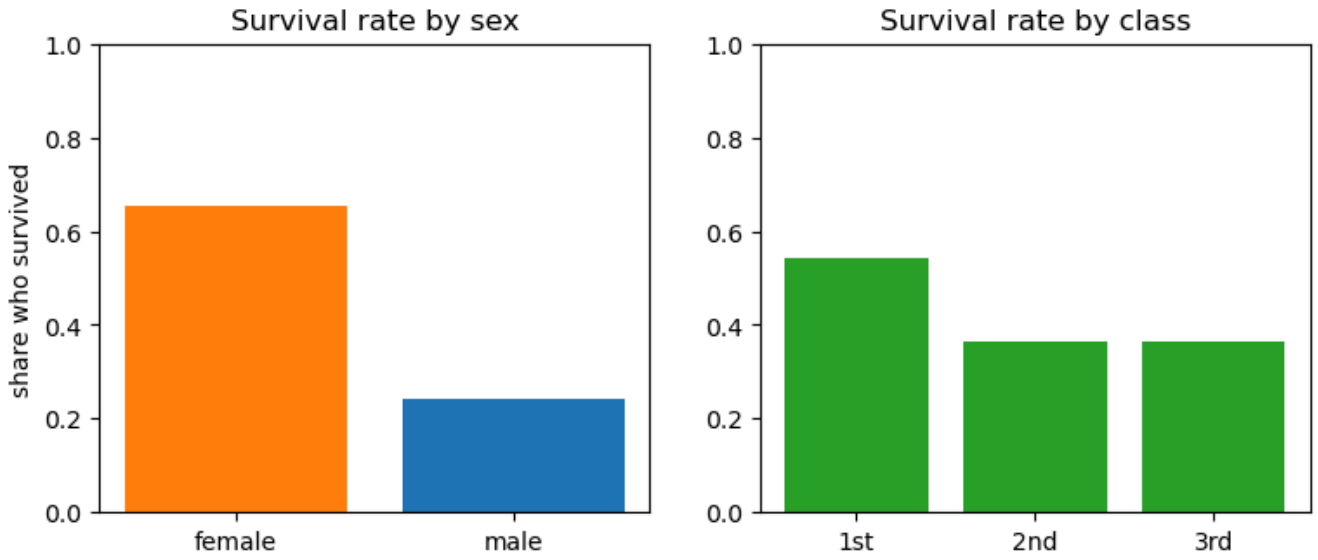
Numbers in a table are exact; a chart makes the difference obvious in one second. First, two bar charts side by side (Day 13). Each chart box ends with `plt.show()`, so the box shows only the chart and no leftover text:

```
import matplotlib.pyplot as plt

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.5))

left.bar(rate_by_sex.index, rate_by_sex.values, color=["tab:orange", "tab:blue"])
left.set_title("Survival rate by sex")
left.set_ylabel("share who survived")
left.set_ylim(0, 1)

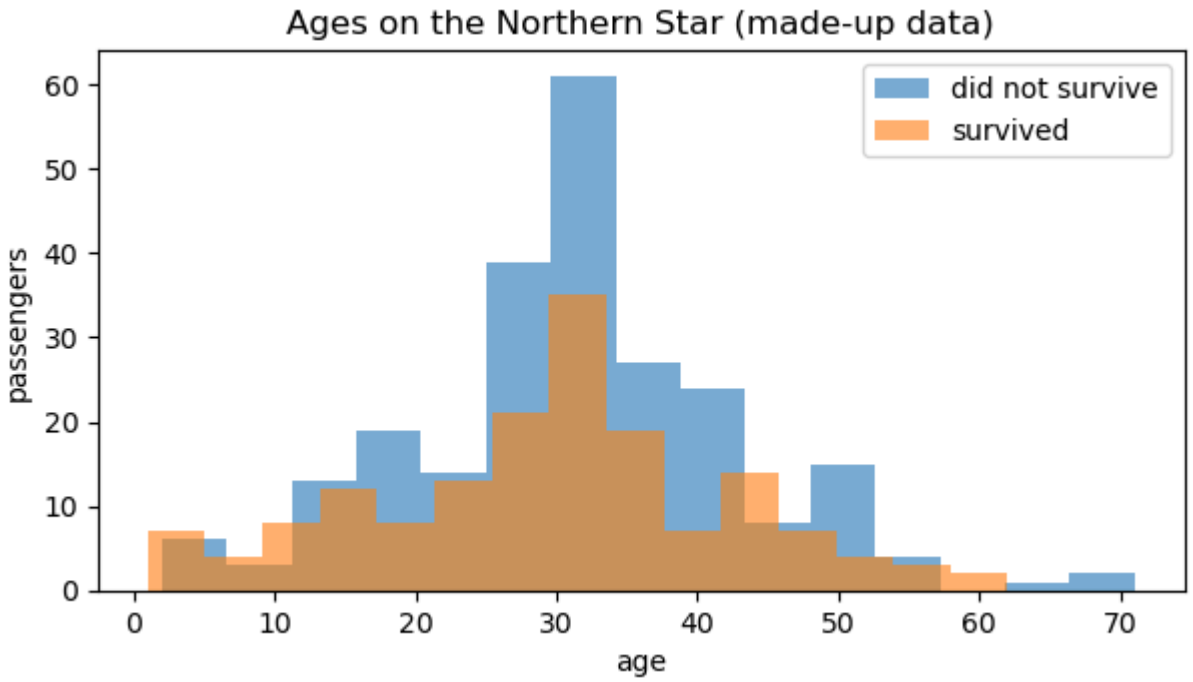
right.bar(["1st", "2nd", "3rd"], by_class["survival_rate"], color="tab:green")
right.set_title("Survival rate by class")
right.set_ylim(0, 1)
plt.show()
```



Next, two histograms on top of each other: the ages of passengers who survived and of those who didn't. `alpha=0.6` makes the bars a bit see-through, so both stay visible:

```
survivors = clean[clean["survived"] == 1]
others = clean[clean["survived"] == 0]

plt.figure(figsize=(7, 3.5))
plt.hist(others["age"], bins=15, alpha=0.6, label="did not survive")
plt.hist(survivors["age"], bins=15, alpha=0.6, label="survived")
plt.title("Ages on the Northern Star (made-up data)")
plt.xlabel("age")
plt.ylabel("passengers")
plt.legend()
plt.show()
```

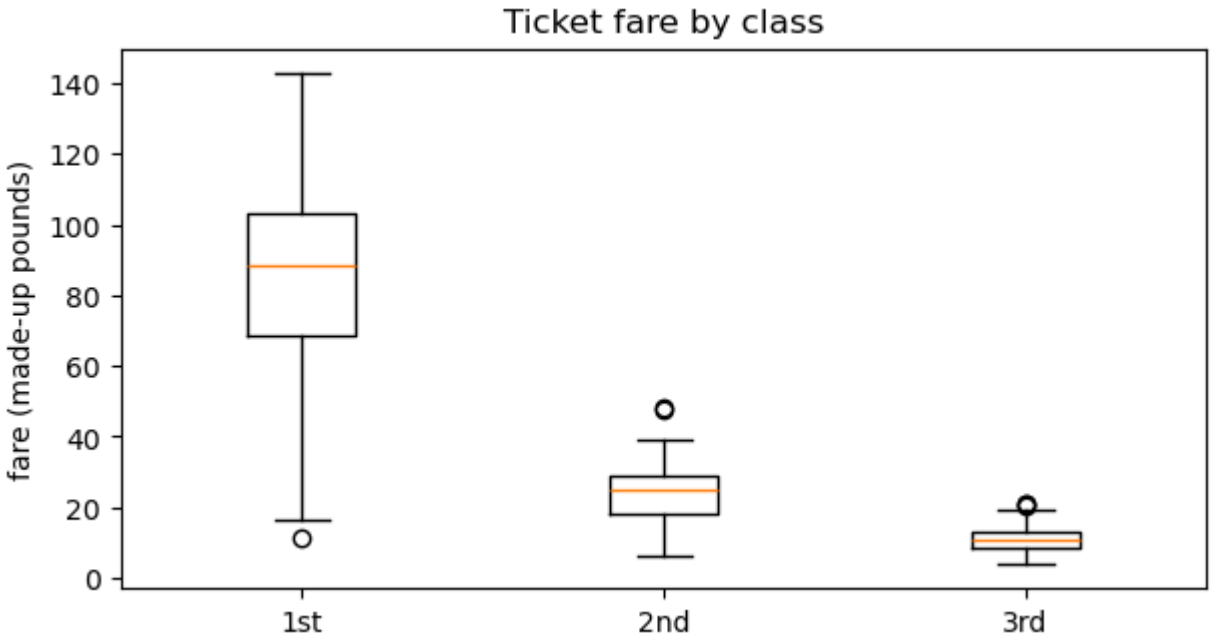


The tall bar near 30 comes partly from our cleaning: the 30 missing ages were all filled with the median, 30. That's a side effect worth knowing about.

Last, a **box plot** of fares per class. It's a new chart type, but it shows what you met on Day 14: the box goes from Q1 to Q3, the line inside is the median, and the dots are outliers.

```
fares = [clean[clean["pclass"] == c]["fare"] for c in [1, 2, 3]]

plt.figure(figsize=(7, 3.5))
plt.boxplot(fares, tick_labels=["1st", "2nd", "3rd"])
plt.title("Ticket fare by class")
plt.ylabel("fare (made-up pounds)")
plt.show()
```



📝 Step 7: Findings

The last step is to answer the questions in plain sentences that someone who never saw the code can understand. Building them from the variables means they stay correct if the data changes:

```
best = table.stack().idxmax()     # the (sex, class) pair with the highest rate
worst = table.stack().idxmin()    # ... and with the lowest

women = rate_by_sex["female"]
men = rate_by_sex["male"]
first = by_class.loc[1, "survival_rate"]
third = by_class.loc[3, "survival_rate"]
children = by_age.loc["child", "mean"]

print(f"1. Overall, {overall_rate:.0%} of the {len(clean)} passengers survived.")
print(f"2. Women survived far more often: {women:.0%} vs {men:.0%} for men.")
print(f"3. First class did best ({first:.0%}); 2nd and 3rd were lower ({third:.0%}).")
print(f"4. Children under 12 survived {children:.0%} of the time.")
print(f"5. Best group: {best[0]}s in class {best[1]}.")
print(f"   Worst group: {worst[0]}s in class {worst[1]}.")
```

```
1. Overall, 41% of the 400 passengers survived.
2. Women survived far more often: 65% vs 24% for men.
3. First class did best (54%); 2nd and 3rd were lower (36%).
4. Children under 12 survived 62% of the time.
5. Best group: females in class 1.
   Worst group: males in class 3.
```

`table.stack()` turns the pivot table into one long column whose labels are (sex, class) pairs, and `idxmax()` gives the *label* of the biggest value (like `argmax` from Day 9, but it returns the name instead of the position).

Careful with "why": the data shows *that* women and first-class passengers survived more often. It doesn't prove *why* (Day 14: correlation is not causation). In our made-up ship we know the reason, because we wrote it into the `chance` line of the first box. In real data you never get to read that line.

⚠️ Common mistakes

- Calculating before cleaning: duplicated rows count some passengers twice.
- Forgetting that the mean of a 0/1 column is a *share* (0.41), not a count. Multiply by 100 or use `:.0%` for a percentage.
- Misspelling a column (`"class"` instead of `"pclass"`) → `KeyError`.
- Trusting a rate from a tiny group (4 seniors) as much as one from 259 adults.
- Saying one thing *causes* another just because they go together.

✅ Recap

- A mini-project follows the EDA routine: questions → inspect → clean → calculate → chart → findings.
- `drop_duplicates()` and `fillna(median)` fix the two classic glitches.
- The mean of a 0/1 column is a rate; `groupby(...)["survived"].mean()` gives a rate per group.
- `pd.cut(column, bins=..., labels=...)` sorts numbers into named ranges.
- `pivot_table` compares two columns at once; `idxmax()` finds the label of the biggest value.
- Findings are plain sentences, built from variables, and careful about cause and effect.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Clean the passenger list

The starter contains a made-up list of 8 passengers with the same two glitches as the lesson: one row appears twice and some ages are missing. Make a DataFrame called `clean` with the duplicate **removed** and the missing ages **filled with the median age**.

```
import pandas as pd
import numpy as np

passengers = pd.DataFrame({
    "passenger_id": [1, 2, 3, 4, 5, 6, 7, 3],
    "sex": ["female", "male", "male", "female", "male", "female", "male",
"male"],
    "age": [29, np.nan, 41, 8, np.nan, 35, 22, 41],
    "survived": [1, 0, 0, 1, 0, 1, 1, 0],
})

# Make clean: no duplicates, missing ages filled with the median
```

Hints:

1. Start with `clean = passengers.drop_duplicates()`.
2. Calculate the median *after* removing duplicates: `clean["age"].median()`.
3. `clean["age"] = clean["age"].fillna(clean["age"].median())`

2. Survival rate by sex

Using the made-up sample of 10 passengers, store the survival rate of each sex in a variable called `rate_by_sex` (one value for `"female"`, one for `"male"`). Use `groupby`; don't count by hand.

```
import pandas as pd

sample = pd.DataFrame({
    "sex": ["female", "male", "male", "female", "male",
            "female", "male", "male", "female", "male"],
    "pclass": [1, 3, 2, 3, 1, 2, 3, 3, 1, 2],
    "survived": [1, 0, 0, 1, 1, 1, 0, 0, 0, 0],
})

# Store the survival rate per sex in rate_by_sex
```

Hints:

1. The mean of a 0/1 column is the survival rate.
2. Group by `"sex"`, pick the `"survived"` column, then take the mean.
3. `rate_by_sex = sample.groupby("sex")["survived"].mean()`

3. Which class did best?

With the same sample, store the survival rate per class in `rate_by_class`. Then store the number of the class with the highest rate in `best_class` (it should be `1`, `2` or `3`).

```
import pandas as pd

sample = pd.DataFrame({
    "sex": ["female", "male", "male", "female", "male",
            "female", "male", "male", "female", "male"],
    "pclass": [1, 3, 2, 3, 1, 2, 3, 3, 1, 2],
    "survived": [1, 0, 0, 1, 1, 1, 0, 0, 0, 0],
})

# rate_by_class = ...
# best_class = ...
```

Hints:

1. Same as exercise 2, but group by `"pclass"`.
2. `idxmax()` gives the label (here the class number) of the biggest value.
3. `best_class = rate_by_class.idxmax()`

4. Were children safer?

Store the survival rate of passengers **under 12** in `child_rate` and of everyone **12 or older** in `adult_rate`. Then print a line like `Children: 67%, others: 43%` (use `:.0%`).

```
import pandas as pd

kids = pd.DataFrame({
    "age": [4, 35, 9, 52, 27, 11, 40, 19, 63, 30],
    "survived": [1, 0, 1, 1, 0, 0, 1, 0, 0, 1],
})

# child_rate = ...
# adult_rate = ...
```

Hints:

1. Filter first: `kids[kids["age"] < 12]` keeps only the children.
2. Then take the mean of their `"survived"` column. For the others use `>= 12`.
3. `print(f"Children: {child_rate:.0%}, others: {adult_rate:.0%}")`

5. Challenge: best and worst group

Back to the full *Northern Star* list. The starter builds and cleans it for you. Find the combination of **sex and class** with the highest survival rate and the one with the lowest. Store them as pairs in `best_group` and `worst_group`, for example `("female", 1)`.

Your code should still work if the data changes, so let pandas find the answer.

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
clean = ships.drop_duplicates()
# --- End of the data ---

# best_group = ...
# worst_group = ...
```

Hints:

1. You can group by two columns at once: `clean.groupby(["sex", "pclass"])["survived"].mean()`. Its labels are (sex, class) pairs.
2. `idxmax()` gives the label of the highest value, `idxmin()` of the lowest.
3. `rates = clean.groupby(["sex", "pclass"])["survived"].mean()`, then `best_group = rates.idxmax()` and `worst_group = rates.idxmin()`.

➡️ Tomorrow

What is machine learning? Today *you* found the patterns in the data. Tomorrow you'll start the Machine Learning phase and let the computer find patterns by itself: you'll learn what *features* and *labels* are, split data into a training part and a test part, and train your very first model with scikit-learn.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Clean the passenger list

```
import pandas as pd
import numpy as np

passengers = pd.DataFrame({
    "passenger_id": [1, 2, 3, 4, 5, 6, 7, 3],
    "sex": ["female", "male", "male", "female", "male", "female", "male",
"male"],
    "age": [29, np.nan, 41, 8, np.nan, 35, 22, 41],
    "survived": [1, 0, 0, 1, 0, 1, 1, 0],
})

clean = passengers.drop_duplicates()
clean["age"] = clean["age"].fillna(clean["age"].median())
```

2. Survival rate by sex

```
import pandas as pd

sample = pd.DataFrame({
    "sex": ["female", "male", "male", "female", "male",
            "female", "male", "male", "female", "male"],
    "pclass": [1, 3, 2, 3, 1, 2, 3, 3, 1, 2],
    "survived": [1, 0, 0, 1, 1, 1, 0, 0, 0, 0],
})

rate_by_sex = sample.groupby("sex")["survived"].mean()
print(rate_by_sex)
```

3. Which class did best?

```
import pandas as pd

sample = pd.DataFrame({
    "sex": ["female", "male", "male", "female", "male",
            "female", "male", "male", "female", "male"],
    "pclass": [1, 3, 2, 3, 1, 2, 3, 3, 1, 2],
    "survived": [1, 0, 0, 1, 1, 1, 0, 0, 0, 0],
})

rate_by_class = sample.groupby("pclass")["survived"].mean()
best_class = rate_by_class.idxmax()
print(rate_by_class)
print("Best class:", best_class)
```

4. Were children safer?

```
import pandas as pd

kids = pd.DataFrame({
    "age": [4, 35, 9, 52, 27, 11, 40, 19, 63, 30],
    "survived": [1, 0, 1, 1, 0, 0, 1, 0, 0, 1],
})

child_rate = kids[kids["age"] < 12]["survived"].mean()
adult_rate = kids[kids["age"] >= 12]["survived"].mean()
print(f"Children: {child_rate:.0%}, others: {adult_rate:.0%}")
```

5. Challenge: best and worst group

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
clean = ships.drop_duplicates()
# --- End of the data ---

rates = clean.groupby(["sex", "pclass"])["survived"].mean()
best_group = rates.idxmax()
worst_group = rates.idxmin()
print("Best:", best_group, "Worst:", worst_group)
```